ПРАКТИЧЕСКАЯ РАБОТА № 4

Автоматизация проверки расписания сайта api.nntu.ru средствами Selenium и pytest

1 ЦЕЛЬ РАБОТЫ

Требуется автоматизировать проверку расписания группы на сайте `https://api.nntu.ru/raspisanie` средствами Selenium и pytest. Скрипт открывает страницу, выбирает кафедру и группу, задаёт диапазон дат, собирает занятия и сравнивает их с заранее сохранённым эталоном. Если ожидаемое расписание подменено, тесты не проходят.

Задачи работы:

1. Автоматизировать открытие страницы расписания и выбор кафедры и группы.
2. Задать диапазон дат и собрать все занятия со страницы.
3. Сохранить эталон в JSON и сравнить с ним текущее расписание.
4. Продемонстрировать успешный и неуспешный прогоны.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Selenium — библиотека для автоматизации веб-браузеров. Она позволяет программно управлять браузером: открывать страницы, находить элементы, вводить текст, нажимать кнопки, читать содержимое и делать снимки экрана. В Python используется пакет `selenium`, а взаимодействие с конкретным браузером обеспечивает драйвер (для Edge — `msedgedriver`, который Selenium Manager подбирает автоматически).

Таблица 1 – Компоненты Selenium и их применение

Компонент	Назначение	Применение в работе
WebDriver	Управляет браузером и выполняет команды.	`webdriver.Edge(options=options)` в headless-режиме.
Локаторы (`By`)	Поиск элементов по id, CSS-селектору, тегу и т. д.	`By.ID` для списков, `By.CSS_SELECTOR` для таблиц.
Ожидания (`WebDriverWait`)	Ожидание появления элемента или условия вместо фиксированных пауз.	Ожидание загрузки списков и 20+ таблиц расписания.
`Select`	Работа с выпадающими списками `<select>`.	Выбор кафедры и группы по значению `value`.

Компонент	Назначение	Применение в работе
`save_screenshot`	Сохранение снимка страницы.	Скриншот расписания и снимок при расхождении.
JavaScript (`execute_script`)	Выполнение кода на странице, установка значений и генерация событий.	Установка дат «с» и «по» с событием `change`.

Для стабильности теста браузер запускается в headless-режиме с размером окна 1400×1200. Явные ожидания применяются вместо `sleep` там, где это возможно: сайт подгружает списки групп и таблицы расписания асинхронно.

3 ОБЪЕКТ ТЕСТИРОВАНИЯ И ИСХОДНЫЕ ДАННЫЕ

Таблица 2 – Объект тестирования и исходные данные

Параметр	Значение
Сайт	https://api.nntu.ru/raspisanie
Форма обучения (кафедра)	Заочная 3 года 10 месяцев, `value = 3`
Группа	АЗИС 23-1, `value = 817`
Диапазон дат	01.09.2026 — 25.12.2026
Браузер	Microsoft Edge, headless

Диапазон ограничен 25 декабря: запись за 26 декабря находится на границе подгрузки таблиц и при повторных запусках могла отсутствовать, что делало тест нестабильным. Эталон снят на устойчивом диапазоне и содержит 85 занятий по 23 датам.

Кнопка применения фильтра на сайте не подключена — обработчик нажатия закомментирован в исходном коде страницы. Поэтому даты устанавливаются через JavaScript с генерацией события `change`. Это обходит дефект сайта, не изменяя проверяемую логику отображения расписания.

4 ЭТАЛОННОЕ РАСПИСАНИЕ

Эталон `expected_schedule.json` получен однократным прогоном того же парсера и проверен вручную. Формат: метаданные (группа, идентификаторы, диапазон дат) и массив занятий. Каждое занятие описывается датой, номером пары, дисциплиной, видом занятия, преподавателем и аудиторией.

```
{
  "group": "АЗИС 23-1",
  "department_id": "3",
  "group_id": "817",
  "date_from": "2026-09-01",
  "date_to": "2026-12-25",
  "entries": [
    {
      "date": "2026-09-07",
      "para": "1 пара",
      "subject": "Организационно-экономическое обоснование научных и технических
решений",
      "type": "Лк.",
      "teacher": "Гусева Ирина Борисовна",
      "room": "218"
    }
  ]
}
```


Ключом сравнения служит пара `(date, para)`: в расписании группы на одну дату приходится одна пара, поэтому такой ключ однозначно идентифицирует занятие. Для негативного сценария подготовлен файл `expected_schedule_mutated.json`, в котором у занятия 07.09.2026 первая пара заменена на «ИЗМЕНЁННЫЙ ПРЕДМЕТ (подмена эталона)». Остальные 84 записи совпадают с реальными данными.

5 РЕАЛИЗАЦИЯ ТЕСТА

Тест состоит из трёх частей: фикстура драйвера, фикстура сбора данных со страницы и три проверки. Полный код приведён в приложении; ниже — ключевые фрагменты.

5.1 Запуск браузера

```
@pytest.fixture(scope="module")
def driver():
    options = Options()
    options.add_argument("--headless=new")
    options.add_argument("--window-size=1400,1200")
    options.add_argument("--disable-gpu")
    options.add_argument("--no-sandbox")

    browser = webdriver.Edge(options=options)
    yield browser
    browser.quit()
```

Драйвер создаётся один раз на модуль: все три теста используют одну и ту же страницу, поэтому повторно открывать браузер и заново выбирать группу не нужно.

5.2 Выбор группы и диапазона дат

```
department = wait.until(
    lambda d: d.find_element(By.ID, "studentAdvert__controls--department")
)
wait.until(lambda d: len(department.find_elements(By.TAG_NAME, "option")) > 1)
Select(department).select_by_value(DEPARTMENT_ID)
time.sleep(2)

groups = wait.until(
    lambda d: d.find_element(By.ID, "studentAdvert__controls--groups")
)
wait.until(lambda d: len(groups.find_elements(By.TAG_NAME, "option")) > 1)
Select(groups).select_by_value(GROUP_ID)
time.sleep(1)

driver.execute_script("""
    const from = document.getElementById('dateBefore');
    const to = document.getElementById('dateAfter');
    if (from) { from.value = arguments[0]; from.dispatchEvent(new Event('change',
{bubbles: true})); }
    if (to)   { to.value   = arguments[1]; to.dispatchEvent(new Event('change',
{bubbles: true})); }
""", DATE_FROM, DATE_TO)

wait.until(lambda d: len(d.find_elements(By.CSS_SELECTOR, "#printable table.raspTable"))
>= 20)
time.sleep(2)
```

После выбора кафедры список групп перезагружается, поэтому перед выбором группы ожидается появление более одной опции. Затем устанавливаются даты и ожидается загрузка не менее 20 таблиц — это признак того, что расписание подгрузилось полностью.

5.3 Разбор страницы

```
def parse_header(text: str):
    """'Понедельник, 7 Сентября 2026' -> '2026-09-07'."""
    match = re.search(r"(\d{1,2})\s+([А-Яа-яЁё]+)\s+(\d{4})", text)
    ...
    return f"{year:04d}-{MONTHS[month]:02d}-{day:02d}"


def read_schedule(driver):
    entries = []
    tables = driver.find_elements(By.CSS_SELECTOR, "#printable table.raspTable")
    for table in tables:
        headers = table.find_elements(By.CSS_SELECTOR, "thead h3")
        date = parse_header(headers[0].text) if headers else None
        if date is None:
            continue
        for row in table.find_elements(By.CSS_SELECTOR, "tbody tr"):
            cells = [c.text.strip() for c in row.find_elements(By.TAG_NAME, "td")]
            if not cells or not cells[0]:
                continue
            subject, ptype = normalize_subject(cells[1] if len(cells) > 1 else "")
            entries.append({
                "date": date,
                "para": cells[0].split(" / ")[0].strip(),
                "subject": subject,
                "type": ptype,
                "teacher": cells[2].strip() if len(cells) > 2 else "",
                "room": cells[3].strip() if len(cells) > 3 else "",
            })
    return entries
```

Заголовок таблицы содержит дату в формате «Понедельник, 7 Сентября 2026». Функция `parse_header` приводит её к ISO-виду `2026-09-07`. Дисциплина на странице записана как «Предмет / Вид занятия», поэтому `normalize_subject` разделяет её на предмет и тип, корректно обрабатывая вариант «Практика /» с пустым типом.

5.4 Сравнение с эталоном

```
expected = json.loads(EXPECTED.read_text(encoding="utf-8"))["entries"]

actual_map = {entry_key(e): e for e in page_entries}
expected_map = {entry_key(e): e for e in expected}

missing = sorted(set(expected_map) - set(actual_map))
extra = sorted(set(actual_map) - set(expected_map))
changed = sorted(
    key for key in set(actual_map) & set(expected_map)
    if actual_map[key] != expected_map[key]
)

if missing or extra or changed:
    driver.save_screenshot(str(SCREENSHOTS / "schedule_mismatch.png"))
    ...
    pytest.fail(
        "Расписание не совпало с эталоном "
        f"(нет: {len(missing)}, лишних: {len(extra)}, изменено: {len(changed)})"
    )
```

Сравнение разделяет расхождения на три категории: отсутствующие на странице занятия, лишние занятия и изменённые поля. Это делает диагностику точной: видно не только факт несовпадения, но и какое именно занятие и какое поле отличаются. При расхождении сохраняется снимок экрана, а в сообщение об ошибке выводятся первые пять расхождений.

Три теста проверяют: выбранную группу, непустоту расписания (не менее 50 занятий и 10 дат) и полное совпадение с эталоном. Разделение важно: если сломается выбор группы, будет понятно, что именно перестало работать, а не просто «расписание не совпало».

6 ПОЗИТИВНЫЙ ПРОГОН

Команда запуска:

```
python -m pytest test_schedule.py -v --tb=short
```

Результат: **3 passed** за 26,1 с. Тест выбрал кафедру и группу, загрузил 85 занятий по 23 датам и подтвердил полное совпадение с эталоном.

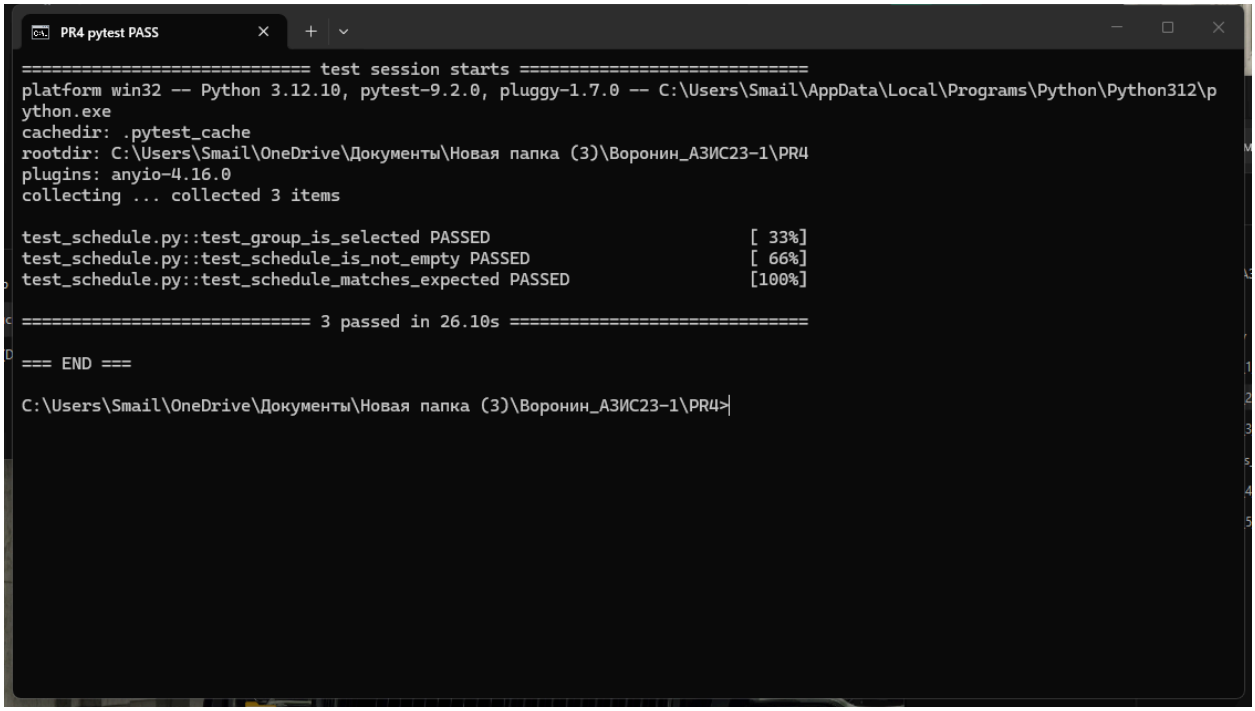


Рисунок 1 – Позитивный прогон: все три теста пройдены

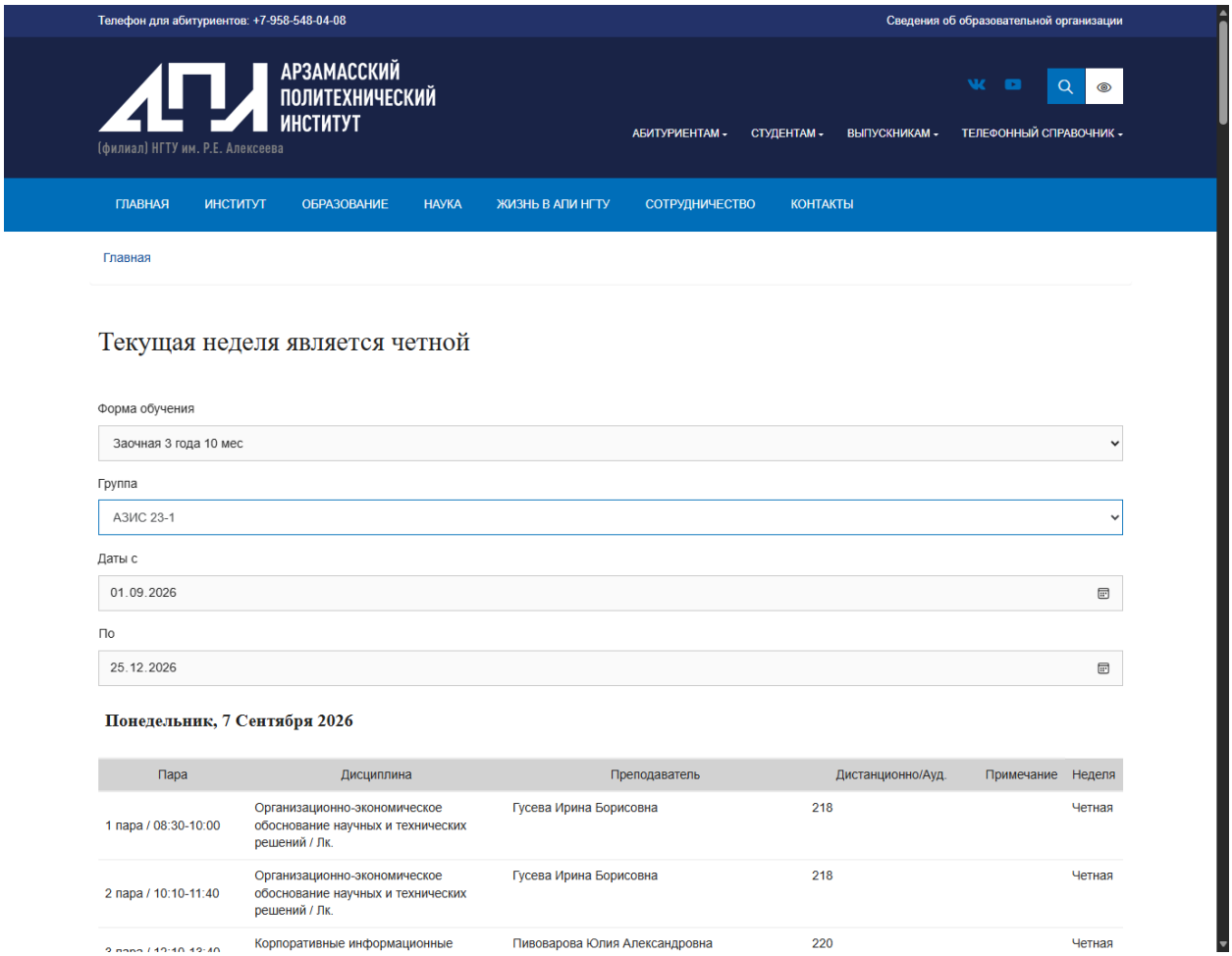


Рисунок 2 – Страница расписания группы АЗИС 23-1, диапазон 01.09.2026–25.12.2026

Таблица 3 – Позитивный прогон

Показатель	Значение
Выбранная группа	АЗИС 23-1
Количество занятий	85
Количество дат	23
Первая дата в эталоне	07.09.2026
Последняя дата в эталоне	13.11.2026
Результат сравнения	Расхождений нет

7 НЕГАТИВНЫЙ ПРОГОН: ПОДМЕНА ЭТАЛОНА

Чтобы показать, что тест действительно проверяет расписание, а не просто открывает страницу, эталон заменяется файлом `expected_schedule_mutated.json`. Путь к эталону задаётся переменной окружения `EXPECTED_SCHEDULE`, поэтому код теста менять не требуется:

```
set EXPECTED_SCHEDULE=expected_schedule_mutated.json
python -m pytest test_schedule.py -v --tb=short
```

Результат: **1 failed, 2 passed** за 27,3 с. Первые два теста (выбор группы и непустота расписания) проходят, потому что страница действительно содержит корректные данные. Третий тест обнаруживает ровно одно изменённое занятие и завершается ошибкой.

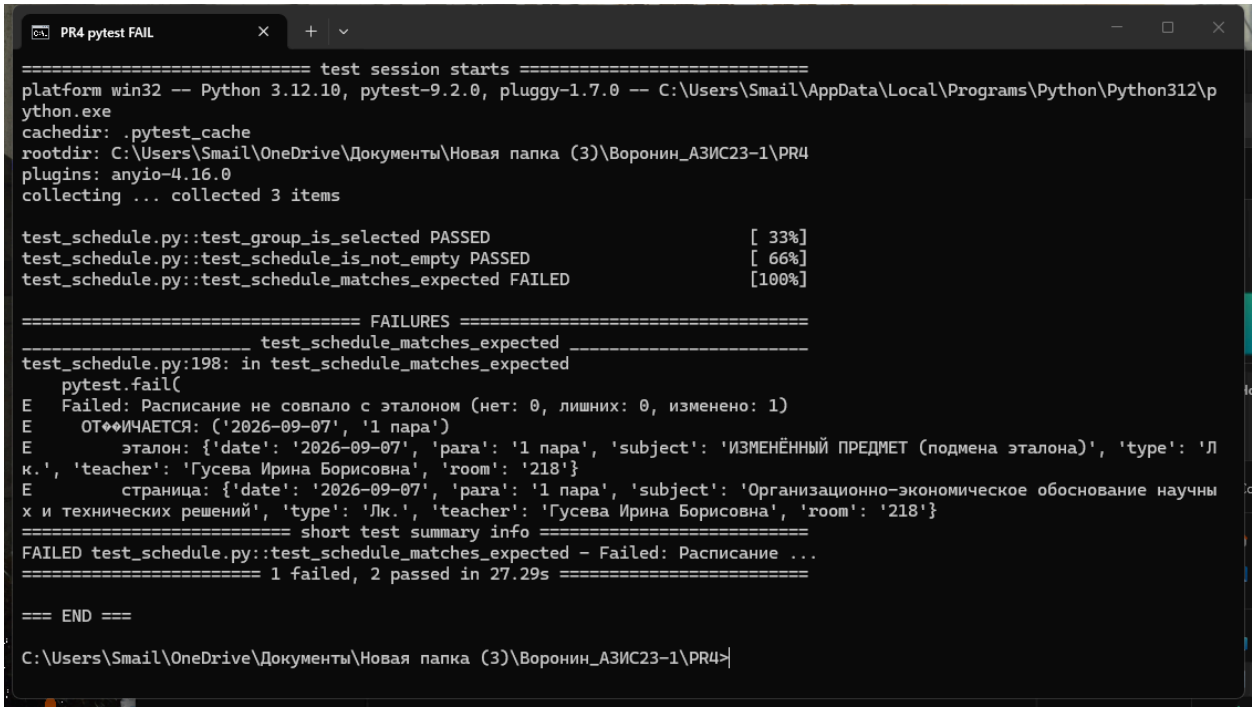


Рисунок 3 – Негативный прогон: подменённый эталон приводит к падению теста

```
Failed: Расписание не совпало с эталоном (нет: 0, лишних: 0, изменено: 1)

ОТЛИЧАЕТСЯ: ('2026-09-07', '1 пара')
    эталон:  {'subject': 'ИЗМЕНЁННЫЙ ПРЕДМЕТ (подмена эталона)', 'type': 'Лк.', ...}
    страница: {'subject': 'Организационно-экономическое обоснование научных и
технических решений', ...}
```

Сообщение показывает и ключ занятия, и оба значения — ожидаемое и фактическое. Этого достаточно, чтобы понять причину падения без повторного запуска. Снимок `schedule_mismatch.png` сохраняется автоматически в момент расхождения.

Таблица 4 – Негативный прогон: подмена эталона

Сценарий	Эталон	Результат
Позитивный	`expected_schedule.json`	3 passed
Негативный	`expected_schedule_mutated.json`	1 failed, 2 passed

8 ВЫВОД

Разработан Selenium-тест, который открывает сайт расписания, выбирает кафедру и группу АЗИС 23-1, задаёт диапазон дат, разбирает все таблицы и сравнивает 85 занятий по 23 датам с заранее сохранённым эталоном. Позитивный прогон завершился результатом 3 passed. При подмене эталона третий тест падает с точным указанием изменённого занятия, что подтверждает: тест проверяет именно соответствие расписания, а не только доступность страницы. Дополнительно выявлен дефект сайта — неработающая кнопка применения фильтра дат, обойдённый установкой значений через JavaScript.

ПРИЛОЖЕНИЕ А. ПОЛНЫЙ КОД ТЕСТА

```
"""ПР4. Selenium-тест расписания группы АЗИС 23-1 на сайте api.nntu.ru.

Тест открывает страницу расписания, выбирает кафедру и группу, задаёт
диапазон дат, разбирает все таблицы расписания и сравнивает результат
с заранее сохранённым эталоном ``expected_schedule.json``.

Запуск (позитивный сценарий):
    pytest test_schedule.py -v

Запуск с подменённым эталоном (негативный сценарий):
    set EXPECTED_SCHEDULE=expected_schedule_mutated.json
    pytest test_schedule.py -v
"""

import json
import os
import re
import time
from pathlib import Path

import pytest
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.edge.options import Options
from selenium.webdriver.support.ui import Select, WebDriverWait

BASE_URL = "https://api.nntu.ru/raspisanie"
DEPARTMENT_ID = "3"          # Заочная 3 года 10 месяцев
GROUP_ID = "817"             # АЗИС 23-1
GROUP_NAME = "АЗИС 23-1"
DATE_FROM = "2026-09-01"
DATE_TO = "2026-12-25"

HERE = Path(__file__).resolve().parent
SCREENSHOTS = HERE / "screenshots"
EXPECTED = Path(os.environ.get("EXPECTED_SCHEDULE", HERE / "expected_schedule.json"))

MONTHS = {
    "января": 1, "февраля": 2, "марта": 3, "апреля": 4, "мая": 5, "июня": 6,
    "июля": 7, "августа": 8, "сентября": 9, "октября": 10, "ноября": 11,
    "декабря": 12,
}


# ---------------------------------------------------------------------------
# Разбор данных страницы
# ---------------------------------------------------------------------------
def parse_header(text: str):
    """'Понедельник, 7 Сентября 2026' -> '2026-09-07'."""
    match = re.search(r"(\d{1,2})\s+([А-Яа-яЁё]+)\s+(\d{4})", text)
    if not match:
        return None
    day, month, year = int(match.group(1)), match.group(2).lower(), int(match.group(3))
    if month not in MONTHS:
        return None
    return f"{year:04d}-{MONTHS[month]:02d}-{day:02d}"


def normalize_subject(cell: str):
    """'Предмет / Экз.' -> ('Предмет', 'Экз.'); 'Практика /' -> ('Практика', '')."""
    cell = " ".join(cell.split())
    if " / " in cell:
        subject, ptype = cell.rsplit(" / ", 1)
    elif cell.endswith(" /"):
        subject, ptype = cell[:-2], ""
    else:
        subject, ptype = cell, ""
    return subject.strip(" /"), ptype.strip()
```

```
def read_schedule(driver):
    """Собирает все занятия со страницы в список словарей."""
    entries = []
    tables = driver.find_elements(By.CSS_SELECTOR, "#printable table.raspTable")
    for table in tables:
        headers = table.find_elements(By.CSS_SELECTOR, "thead h3")
        date = parse_header(headers[0].text) if headers else None
        if date is None:
            continue
        for row in table.find_elements(By.CSS_SELECTOR, "tbody tr"):
            cells = [c.text.strip() for c in row.find_elements(By.TAG_NAME, "td")]
            if not cells or not cells[0]:
                continue
            subject, ptype = normalize_subject(cells[1] if len(cells) > 1 else "")
            entries.append({
                "date": date,
                "para": cells[0].split(" / ")[0].strip(),
                "subject": subject,
                "type": ptype,
                "teacher": cells[2].strip() if len(cells) > 2 else "",
                "room": cells[3].strip() if len(cells) > 3 else "",
            })
    return entries


def entry_key(entry):
    """Ключ занятия: в расписании группы на одну дату приходится одна пара."""
    return (entry["date"], entry["para"])


# ---------------------------------------------------------------------------
# Фикстуры
# ---------------------------------------------------------------------------
@pytest.fixture(scope="module")
def driver():
    options = Options()
    options.add_argument("--headless=new")
    options.add_argument("--window-size=1400,1200")
    options.add_argument("--disable-gpu")
    options.add_argument("--no-sandbox")

    browser = webdriver.Edge(options=options)
    yield browser
    browser.quit()


@pytest.fixture(scope="module")
def page_entries(driver):
    """Открывает расписание, выбирает группу и возвращает разобранные занятия."""
    SCREENSHOTS.mkdir(exist_ok=True)
    driver.get(BASE_URL)
    wait = WebDriverWait(driver, 40)

    department = wait.until(
        lambda d: d.find_element(By.ID, "studentAdvert__controls--department")
    )
    wait.until(lambda d: len(department.find_elements(By.TAG_NAME, "option")) > 1)
    Select(department).select_by_value(DEPARTMENT_ID)
    time.sleep(2)

    groups = wait.until(
        lambda d: d.find_element(By.ID, "studentAdvert__controls--groups")
    )
    wait.until(lambda d: len(groups.find_elements(By.TAG_NAME, "option")) > 1)
    Select(groups).select_by_value(GROUP_ID)
    time.sleep(1)

    driver.execute_script(
        """
        const from = document.getElementById('dateBefore');
        const to = document.getElementById('dateAfter');
        if (from) { from.value = arguments[0]; from.dispatchEvent(new Event('change',
{bubbles: true})); }
```

```
        if (to)   { to.value   = arguments[1]; to.dispatchEvent(new Event('change',
{bubbles: true})); }
        """,
        DATE_FROM,
        DATE_TO,
    )
    # Ждём, пока подгрузятся все таблицы расписания.
    wait.until(lambda d: len(d.find_elements(By.CSS_SELECTOR, "#printable
table.raspTable")) >= 20)
    time.sleep(2)

    driver.save_screenshot(str(SCREENSHOTS / "schedule_page.png"))
    entries = read_schedule(driver)
    assert entries, "На странице не найдено ни одной таблицы с занятиями"
    return entries


# ---------------------------------------------------------------------------
# Тесты
# ---------------------------------------------------------------------------
def test_group_is_selected(driver, page_entries):
    groups = Select(driver.find_element(By.ID, "studentAdvert__controls--groups"))
    assert groups.first_selected_option.text.strip() == GROUP_NAME


def test_schedule_is_not_empty(page_entries):
    dates = {entry["date"] for entry in page_entries}
    assert len(page_entries) >= 50
    assert len(dates) >= 10


def test_schedule_matches_expected(driver, page_entries):
    expected = json.loads(EXPECTED.read_text(encoding="utf-8"))["entries"]

    actual_map = {entry_key(e): e for e in page_entries}
    expected_map = {entry_key(e): e for e in expected}

    missing = sorted(set(expected_map) - set(actual_map))
    extra = sorted(set(actual_map) - set(expected_map))
    changed = sorted(
        key for key in set(actual_map) & set(expected_map)
        if actual_map[key] != expected_map[key]
    )

    if missing or extra or changed:
        driver.save_screenshot(str(SCREENSHOTS / "schedule_mismatch.png"))
        details = []
        for key in missing[:5]:
            details.append(f"  НЕТ НА СТРАНИЦЕ: {key}")
        for key in extra[:5]:
            details.append(f"  ЛИШНЕЕ НА СТРАНИЦЕ: {key}")
        for key in changed[:5]:
            details.append(
                f"  ОТЛИЧАЕТСЯ: {key}\n"
                f"      эталон: {expected_map[key]}\n"
                f"      страница: {actual_map[key]}"
            )
        pytest.fail(
            "Расписание не совпало с эталоном "
            f"(нет: {len(missing)}, лишних: {len(extra)}, изменено: {len(changed)})\n"
            + "\n".join(details)
        )

    assert len(actual_map) == len(expected_map)
```